REMEMBER

1. Believe in yourself, you can easily score 100%
2. Before solving any question, first clear your concepts related to that question.
3. Always try to solve yourself first then check doc solution (while attempting questions 1st time).
4. Practice a lot and try to solve all questions in 2 hrs.

Read the instruction carefully.

1. There will be one controller machine and 5 nodes.
2. You will be provided the root login and root password, and you have to create playbooks via admin user. [Login on controller node with root then switch to admin user]
3. Create all your playbooks in /home/admin/ansible [mkdir /home/admin/ansible]
4. Create config and inventory file in /home/admin/ansible/
5. Create a roles directory in /home/admin/ansible/ [mkdir /home/admin/ansible/roles]

Note: vim command will not be working, so you have to install vim in your controller machine (yum install vim -y) but vi command will be accessible.

First step -> read instructions carefully
Second step -> ssh <given user>@< master/controller node name >
Third key step - > yum install vim -y

(Optional Part just to configure vim in controller node)
In admin home directory to create the .vimrc file

vim /home/admin/.vimrc
set ai
set ts=2
set et
set cursorcolumn

Step1: Read the instructions carefully, there you'll get ip addresses and fqdn of all nodes & root password of controller node. And folder name where you have to put all playbooks and config file. Also username of managed node and some other things.

Remember: - We won't need root password anywhere.
- User name of managed node and controller node would be same.

Step2: ssh to controller node on root account. Then switch user(su - username).

Step3: Install and setup vim, install ansible

Step4: Create folder with that name given in instruction

Step5: Start solving questions

Q1)

Install and configure Ansible

Install and configure Ansible on control node control.sector1.example.com as

follows:Install the required packages

Create a static inventory file called /home/Catherine/ansible/inventory so that:

node1 is a member of the **dev** host group

node2 is a member of the **test** host group

node3 and **node4** are member of the **prod** host group

node5 is a member of the **balancers** host group

The **prod** group is a member of the **webservers** host group

Create a configuration file called **/home/Catherine/ansible/ansible.cfg** so that:

The host inventory file is **/home/Catherine/ansible/inventory**

The default content collections directory is **/home/Catherine/ansible/mycollection**

The default roles directory is **/home/Catherine/ansible/roles**

INSTRUCTION:

For this ques, you'll find hostname in instructions.

After solving, always check:

- Ansible all –m ping (check connectivity)
- Ansible all –a 'id' (check if privilege escalation working or not, its running command with root power or not)

If you are not getting root, then there's issue in ansible.cfg file.

Solution:

yum install ansible-core -y
yum install ansible-navigator -y

mkdir /home/Catherine/ansible/mycollection
mkdir /home/Catherine/ansible/roles
mkdir /home/Catherine/ansible

vim /home/Catherine/ansible/inventory

[dev] node1

[test] node2

[prod]
node[3:4] # range method to define the multiple fqdns in single string range

[balancers] node5

[webservers:children] #super group logic given in book prod

vim /home/Catherine/ansible/ansible.cfg

[defaults]
inventory = inventory
roles_path = /home/Catherine/ansible/roles
collections_path = /home/Catherine/ansible/mycollection
remote_user = tom
ask_pass = false
host_key_checking = false

[privilege_escalation]
become = true
become_method = sudo
become_user = root
become_ask_pass = false

NOTE - To get everything automatically in ansible.cfg use -
ansible-config init --disabled > ansible.cfg

Q2. Create and run an Ansible ad-hoc command. As a system administrator, you will need to install software on the managed nodes.
Create a shell script called yum-pack.sh that runs an Ansible ad-hoc command to create a yum repository on each of the managed nodes as follows:
NOTE: you need to create 2 repos (BaseOS and AppStream) in managed node side

i) BaseOS
a. name: baseos
b. baseurl: http://classroom.example.com/content/rhel8.0/x86_64/dvd/BaseOS/
c. description:Base OS Repo
d. gpgcheck: yes
e. enabled: yes
key: http://classroom.example.com/content/rhel8.0/x86_64/dvd/RPM-GPG-KEY-redhat-release

ii) AppStream
a. name: appstream
b. baseurl: http://classroom.example.com/content/rhel8.0/x86_64/dvd/AppStream/
c. description:Appstrean Repo
d. gpgcheck: yes
e. enabled: yes
key: http://classroom.example.com/content/rhel8.0/x86_64/dvd/RPM-GPG-KEY-redhat-release

INSTRUCTIONS:

Solve this question carefully as its base for most of the upcoming questions.
Always check after solving as sometimes we do small mistakes and yum not configure properly and we won't even know. Then we'll get errors in installing packages while running playbooks.

Q. Simplest way to check?

Ans: → ansible all –a 'yum repolist' (use ansible adhoc command to check)
If you're getting packages in BaseOs and Appstream both, then it's fine.

Q. What's "-b" at the end of each command?

Ans. It's for running that command with root power. (We can also remove that as we have already given privilege escalation in inventory file)

Q. What is gpg key?

Ans. It's for verifying the software.
Link: https://www.redhat.com/sysadmin/rpm-gpg-verify-packages

Solution:
vim yum-pack.sh

```
#!/bin/bash

ansible all -m yum_repository -a 'file=external_repo name=baseos description="Base OS Repo" baseurl=http://classroom.example.com/content/rhel8.0/x86_64/dvd/BaseOS/ gpgcheck=yes gpgkey=http://classroom.example.com/content/rhel8.0/x86_64/dvd/RPM-GPG-KEY-redhat-release enabled=yes state=present' -b


ansible all -m yum_repository -a 'file=external_repo name=appstream description="AppStream Repo" baseurl=http://classroom.example.com/content/rhel8.0/x86_64/dvd/AppStream/ gpgcheck=yes gpgkey=http://classroom.example.com/content/rhel8.0/x86_64/dvd/RPM-GPG-KEY-redhat-release enabled=yes state=present' -b
```

chmod +x yum-pack.sh
./yum-pack

Q3. Create a playbook called /home/Catherine/ansible/packages.yml that:
- Installs the php and mariadb packages on hosts in the dev, test, and prod host groups
- Installs the RPM Development Tools package group on hosts in the dev host group
- Updates all packages to the latest version on hosts in the dev host group

INSTRUCTIONS:
Before solving this, let's clear the concept of package groups first.

On rhel, we can either install software (RPM) individually or install related software in group.
Package group contain packages that perform related tasks such as development tools, web servers.

Some command, just to get familiar with package group:

- Yum groups summary
- yum groups list
- yum groups info "RPM Development Tools"
- yum groups install "RPM Development Tools"
- yum groups remove "RPM Development Tools"

Solution:

> vim /home/Catherine/ansible/packages.yml

```
---
- name: Playbook for packages.yml
  hosts: all
  vars:
    pkgs:
      - php
      - mariadb
  tasks:
    - name: install the packages
      yum:
```

```
    name: "{{ item }}"
  state: present
  loop: "{{ pkgs }}"
  when: inventory_hostname in groups['dev'] or inventory_hostname in groups['test'] or
inventory_hostname in groups['prod']

- name: install the RPM development tool package group
  yum:
    name: "@RPM Development tools"
    state: present
  when: inventory_hostname in groups['dev']

- name: update all packages
  yum:
    name: '*'
    state: latest
  when: inventory_hostname in groups['dev']
```

Q4. Install the RHEL system roles package and create a playbook called /home/Catherine/ansible/timesync.yml that:
-- Runs on all managed hosts
-- Uses the timesync role
-- Configures the role to use the time server 172.25.254.250
-- Configuresthe role to set the iburst parameter as enabled

INSTRUCTIONS:

For this ques, you can easily get examples in README.md file, you can copy paste from here:

- cat /usr/share/ansible/roles/rhel-system-roles.timesync/README.md

And don't forget to give full path of role in playbook otherwise it won't work.

For practice in local lab, you have to add one more repo in yum repository for centos repo for installing updated version of system roles:
http://mirror.centos.org/centos/8/AppStream/x86_64/os/

Ques: What are system roles?

Ans: System roles are pre created roles certified by RedHat present in system.
https://www.redhat.com/sysadmin/ansible-system-role

Ques: What is timesync role?

Ans: timesync role installs and configures NTP to be operated as NTP client to synchronize the system clock with NTP server.

Solution:

sudo yum install rhel-system-roles

vim /home/Catherine/ansible/timesync.yml

```
---
- name: playbook for timesync.yml
  hosts: all
  vars:
    timesync_ntp_servers:
      - hostname: 172.25.254.250
        iburst: yes

  roles:
    - /usr/share/ansible/roles/rhel-system-roles.timesync

...
```

Q5. Create a role called apache in /home/Catherine/ansible/roles with the following requirements

- The httpd package is installed, enabled on boot, and started
- The firewall is enabled and running with a rule to allow access to the web server
- A template file index.html.j2 exists (you have to create this file) and is used to create the file /var/www/html/index.html with the following output:

Welcome to HOSTNAME on IPADDRESS

- where HOSTNAME is the fully qualified domain name of the managed node and IPADDRESS is the IP address of the managed node.
- Create a playbook called **/home/Catherine/ansible/newrole.yml** that uses this role as follows:
  * The playbook runs on hosts in the webservers host group

Ques: Do we have to use default_ipv4.address or enp0s3.ipv4.address in exam?
Ans: Use default one.

Ques: In doc, there's two way given to solve this. Which one should I use?
Ans: Try to use 2nd one (which has vars file), because purpose of role is to separate components instead of putting them in one single file. So, try to use separate vars file of role.

Solution:

> ansible-galaxy init --init-path=roles apache

> vim roles/apache/tasks/main.yml

```
- name: install all packages
  yum:
    name: "{{ item }}"
  state: present
  loop: "{{ pkgs }}"

- name: start and enable the services
  service:
    name: "{{ item }}"
  state: started
  enabled:  true
  loop: "{{ pkgs }}"

- name:
  firewalld:
    service: "{{ item }}"
    permanent: true
    immediate: true
    state: enabled
  loop: "{{ rule }}"


- name: create info file from template
  template:
    src: index.html.j2
    dest: /var/www/html/index.html
```

> vim roles/apache/vars/main.yml # for vars for above role task

```
pkgs:
  - httpd
  - firewalld
rule:
  - http
  - https
```

> vim roles/apache/templates/index.html.j2

```
Welcome to {{ ansible_facts['fqdn'] }} on {{ ansible_facts['default_ipv4']['address'] }}
```

> vim apacherole.yml

```
---
- name: playbook for apache role
  hosts: prod
  roles:
        - role: apache
...
```

Q6. Use Ansible Galaxy with a requirements file called /home/Catherine/ansible/roles/requirement.yml to download and install roles to /home/admin/ansible/roles from the following URLs:

-- http://classroom.example.com/content/examfun.tar.gz
The name of this role should be balancer

-- http://classroom.example.com/content/examfun.tar.gz
The name of this role should be phpinfo

Ques: What's that requirement file?

Ans: If have lots of URL to download and install roles, we can put all the URLs in one file and give the name of role. Hence install all the roles at once.
They install roles by default at the role path given in Inventory.

Solution:

vim /home/Catherine/ansible/roles/requirements.yml

```
---
- src: http://classroom.example.com/content/examfun.tar.gz
  name: balancer

- src: http://classroom.example.com/content/examfun.tar.gz
  name: phpinfo

...
```

ansible-galaxy install -r roles/requirement.yml -p roles/

pwd before running the command must be /home/Catherine/ansible

Q7. Create a playbook called balance.yml as follows:
The playbook contains a play that runs on hosts in the balancers host group and uses the balancer role.
- This role configures a service to load balance web server requests between hosts in the webservers host group.
- When implemented, browsing to hosts in the balancers host group (for example http://node5.example.com) should produce the following output:
Welcome to node3.example.com on 192.168.10.z
- Reloading the browser should return output from the alternate web server:
Welcome to node4.example.com on 192.168.10.a

* The playbook contains a play that runs on hosts in the webservers host group and uses the phphello role.
When implemented, browsing to hosts in the webservers host group with the URL /hello.php should produce the following output:
Hello PHP World from FQDN
where FQDN is the fully qualified domain name of the host.
For example, browsing to http://node3.example.com/hello.php, should produce the following output:
Hello PHP World from node3.example.com
along with various details of the PHP configuration including the version of PHP that is installed.

* Similarly, browsing to http://node4.example.com/hello.php, should produce the following output:
Hello PHP World from node4.example.com
along with various details of the PHP configuration including the version of PHP that is installed.

INSTRUCTIONS:
This one is the easiest question. Here you just have to use the roles installed in previous question.

Ques: why we are first running a play on all nodes without any task?
Ans: To gather information of other nodes that is used by balancer role.

Solution:

vim balance.yml

```
---
- hosts: all
  task: []
```

```
- name: play for balancer group
  hosts: balancers
  roles:
        - balancer

- name: play for webserver group
  hosts: webservers
  roles:
        - phpinfo
...
```

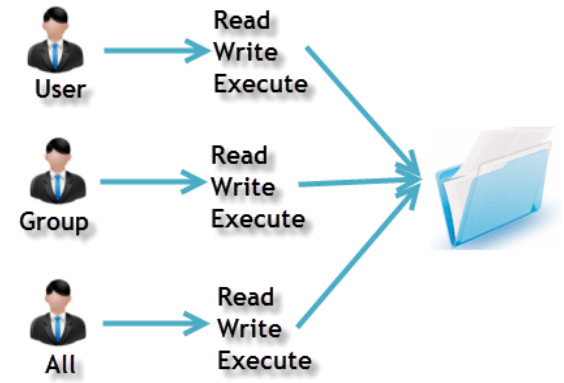
Q8. Create a playbook called web.yml as follows:
* The playbook runs on managed nodes in the dev host group
* Create the directory /webdev with the following requirements:
- membership in the webdev group
* regular permissions: owner=read+write+execute, group=read+write+execute, other=read+execute
special permissions: set group ID
* Symbolically link /var/www/html/webdev to /webdev
* Create the file /webdev/index.html with a single line of text that reads: Development

INSTRUCTIONS:

For this question, first clear you concept about permissions and SELinux.

Octal	Decimal	Permission	Representation
000	0 (0+0+0)	No Permission	---
001	1 (0+0+1)	Execute	--x
010	2 (0+2+0)	Write	-w-
011	3 (0+2+1)	Write + Execute	-wx
100	4 (4+0+0)	Read	r--
101	5 (4+0+1)	Read + Execute	r-x
110	6 (4+2+0)	Read + Write	rw-
111	7 (4+2+1)	Read + Write + Execute	rwx

Owners assigned Permission On Every File and Directory



In this ques, 1st we have to create directory with given permission then asked to create shortcut of folder called symbolic link. So we have to create one more source for this destination.
Then it's asked to create a file index.html in /webdev folder that is linked to /var/www/html.

But we can't disable SELinux in exam. Here we're creating index.html in other folder (/webdev) linked to document root (/var/www/html). SELinux won't allow this (because /var/www/html don't have access to file of some other folder because of SELinux).

That's why we are setting permission – setype: "httpd_sys_content_t"
Due to this permission, apache webserver is using this folder. So same permission we are giving to /webdev/index.html

Don't forget to give ownership of apache user.
After solving this ques always check using curl.

> Curl node1.example.com/webdev/ (It should print Development)

Q. What is httpd_sys_content_t?
Ans: httpd_sys_content_t - > this is the permission of apache webserver.

Q. What if I forget this permission in exam?
Ans: Go to the node where httpd is installed. Check permission of document root.

-> ls -Z -d /var/www/html

Ques: Why given 02775?
Ans: 0 - > we are adding leading 0 so that ansible know it's it's an octal number. You will get more information about this in "ansible-doc file". In argument – "mode".

2 - > set GID permission
7 - > Owner(read + write + execute)
7 - > Group(read + write + execute)

5 - > Others(read + execute)

Ques: What is GID?

Ans: Groups in Linux are defined by GID (group id).

When set group ID permission is applied to a directory, files that were created in this directory belong to the group to which the directory belongs. Any user who has write or execute permission in the directory can create a file here. However the files belongs to the groups that owns the directory, not to the user's group ownership.

Hence Files in that directory will have the same group as the group of parent directory.

Solution:

vim web.yml

```
---
- hosts: prod
  tasks:

      - group:
            name: webdev

      - file:
            state: directory
            path: /webdev
            mode: 02775
            group: webdev
            owner: apache
            setype: "httpd_sys_content_t"

      - file:
            src: /webdev
            path: /var/www/html/myweb
            state: link
            mode: 02775
            owner: apache
            setype: "httpd_sys_content_t"

      - copy:
            dest: /webdev/index.html
            mode: 0640
            content: "Depolyment"
            owner: apache
            setype: "httpd_sys_content_t"
...
```

Q9. Create an Ansible vault to store user passwords as follows:
* The name of the vault is vault.yml
* The vault contains two variables as follows:
- dev_pass with value wakennym
- mgr_pass with value rocky
* The password to encrypt and decrypt the vault is atenorth
* The password is stored in the file /home/admin/ansible/password.txt

Note: Don't forget to change permission of password.txt (where we store password)

Solution:

vim password.txt
atenorth

ansible-vault create --vault-password-file=password.txt vault.yml
dev_pass: wakennym
mgr_pass: rocky

chmod 0600 password.txt

Q10. Generate a hosts file:
* Download an initial template file called hosts.j2 from http://classroom.example.com/content/hosts.j2 to /home/admin/ansible/. Complete the template so that it can be used to generate a file with a line for each inventory host in the same format as /etc/hosts
* Create a playbook called gen_hosts.yml that uses this template to generate the file /etc/myhosts on hosts in the dev host group.
* When completed, the file /etc/myhosts on hosts in the dev host group should have a line for each managed host:

127.0.0.1 localhost localhost.localdomain localhost4 localhost4.localdomain4
::1 localhost localhost.localdomain localhost6 localhost6.localdomain6

192.168.10.x node1.example.com node1
192.168.10.y node2.example.com node2
192.168.10.z node3.example.com node3
192.168.10.a node4.example.com node4
192.168.10.b node5.example.com node5

INSTRUCTIONS:
For this question, 1st clear the concept of hostvars and groups.
Hostvars, groups, inventory_hostname are some magic variabls in ansible.

- group_names: List of groups the current host is part of
- groups: A dictionary/map with all the groups in inventory and list of hosts belonging to each group.
- hostvars: A dictionary/map with all the hosts in inventory and variables assign to them. With hostvars you can access variables defined for any host in play. ansible_facts is also one of the variable of hostvars, from there we can retrieve everything about that host. Try to explore yourself what all it contains to get comfortable with it.

As this file should have IP, fqdn and hostname of all nodes, therefore first we are running a play on all nodes to gather these information.

Solution:

wget http://classroom.example.com/content/hosts.j2

vim hosts.j2

```
127.0.0.1   localhost localhost.localdomain localhost4 localhost4.localdomain4
::1         localhost localhost.localdomain localhost6 localhost6.localdomain6

{% for x in groups['all'] %}
{{ hostvars[x]['ansible_facts']['default_ipv4']['address'] }} {{
hostvars[x]['ansible_facts']['fqdn'] }} {{ hostvars[x]['ansible_facts']['hostname'] }}
{% endfor %}
```

vim gen_hosts.yml

```
- hosts: all
  tasks: []

- hosts: dev
  tasks:
        - template:
            src: "/home/user/anisble/hostdetails.j2"
            dest: "/etc/myhosts"
```

Q11. Create a playbook called hwreport.yml that produces an output file called /root/hwreport.txt on all managed nodes with the following information:

-- Inventory host name
-- Total memory in MB
-- BIOS version
-- Size of disk device vda

-- Size of disk device vdb
Each line of the output file contains a single keyvalue pair.
* Your playbook should:
-- Download the file hwreport.empty from the URL
http://classroom.example.com/content/hwreport.empty and save it as /root/hwreport.txt
-- Modify with the correct values.
NOTE: If a hardware item does not exist, the associated value should be set to NULL.

For solving this ques, many of us came across two ways and get confused.
Both are correct and you'll get full marks in both.
1st one is easiest and takes less time.
2nd one is little bit lengthy but its exact solution of this ques.

I suggest practice both and use that one in which you are comfortable.
I practiced both but in exam I used 1st approach because of time.

In exam, you'll get hwreport.empty file like this:
-- Inventory host name: hostname
-- Total memory in MB: memory
-- BIOS version: bios_version
-- Size of disk device vda: vda_size
-- Size of disk device vdb: vdb_size

1st sol:
Step1: Download file on controller node
Step2: Copy it in /home/admin/ansible/hwreport.j2
Step3: Create hwreport.yml till rescue – debug (don't add 2nd template task)
Step4: Run the playbook and check errors that we have registered in hwreport variable
Step5: Create copy of hwreport.j2 -> hwreport2.j2
Step6: Modify hwreport2.j2 according to error you're getting.
Step7: Add remaining template task in playbook and run.

Solution:
(Note: remove vdb from serverd - for making the scenerio)

wget http://classroom.example.com/content/hwreport.empty

vim hwreport.j2

-- Inventory host name : {{ ansible_facts['fqdn'] }}
-- Total memory in MB: {{ ansible_facts['memtotal_mb'] }}
-- BIOS version: {{ ansible_facts['bios_version'] }}
-- Size of disk device vda: {{ ansible_facts['devices']['vda']['size'] }}
-- Size of disk device vdb: {{ ansible_facts['devices']['vdb']['size'] }}

vim hwreport2.j2

```
 -- Inventory host name : {{ ansible_facts['fqdn'] }}
 -- Total memory in MB: {{ ansible_facts['memtotal_mb'] }}
 -- BIOS version: {{ ansible_facts['bios_version'] }}
 -- Size of disk device vda: {{ ansible_facts['devices']['vda']['size'] }}
{% if 'AnsibleUndefinedVariable' in hwreport.msg %}
 -- Size of disk device vdb: NULL
{% endif %}
```

vim hwreport.yml

```
---
- name: playbook for hwreport.yml
  hosts: all
  tasks:
    - block:
      - name: generate hardware report
        template:
          src:  hwreport.j2
          dest: /root/hwreport.txt
        register: hwreport

      rescue:
      - debug:
          var: hwreport

      - name: generate hardware report on missing
        template:
          src: hwreport2.j2
          dest: /root/hwreport.txt

...
```

Or

2nd approach:
Step1: Write the playbook to download file in all nodes and run.
Step2: ssh to any node and cat /root/hwreport.txt, copy the content and exit.
Step3: Paste the content at bottom of the playbook.
Step4: Then write further tasks for replacing content using the content we pasted in previous step.

```
---
- name: playbook running on all the nodes
  hosts: all

  vars:
  - destination: "/root/hwreport.txt"

  tasks:
  - name: downloading url
    get_url:
      url: " http://classroom.example.com/content/hwreport.empty "
      dest: "{{ destination }}"

  - name: replacing hostname content
    replace:
     regexp: "hostname"
     path: "{{ destination }}"
     replace: "{{ ansible_hostname }}"

  - name: replacing memory content
    replace:
     regexp: "memory"
     path: "{{ destination }}"
     replace: "{{ ansible_memtotal_mb }}"

  - name: replacing bios version content
    replace:
     regexp: "bios_version"
     path: "{{ destination }}"
     replace: "{{ ansible_bios_version }}"

  - name: replacing sda size content
    replace:
     regexp: "sda_size"
     path: "{{ destination }}"
     replace: "{{ ansible_facts['devices']['sda']['size'] }}"
    register: sda_info
    ignore_errors: yes

  - name: replacing sda size with NONE
    replace:
      regexp: "sda_size"
      path: "{{ destination }}"
      replace: "NONE"
```

```
    when: sda_info.failed == true

  - name: replacing sdb size content
    replace:
     regexp: "sdb_size"
     path: "{{ destination }}"
     replace: "{{ ansible_facts['devices']['sdb']['size'] }}"
    register: sdb_info
    ignore_errors: yes

  - name: replacing sdb size with NONE
    replace:
      regexp: "sdb_size"
      path: "{{ destination }}"
      replace: "NONE"
    when: sdb_info.failed == true
...
```

Q12. Modify file content in all hosts. Create a playbook called /home/Catherine/ansible/issue.yml as follows:
* The playbook runs on all inventory hosts
* The playbook replaces the contents of /etc/issue with a single line of text as follows:
- On hosts in the dev host group, the line reads: Development
- On hosts in the test host group, the line reads: Test
- On hosts in the prod host group, the line reads: Production

Solution:
vim issue.yml

```
---
- name: playbook for issue.yml
  hosts: all
  tasks:
    - name: replace the content in dev group
      copy:
        content: "Development"
        dest: /etc/issue
      when: inventory_hostname in groups['dev']

    - name: replace the content in test group
      copy:
        content: "Test"
```

```
      dest: /etc/issue
    when: inventory_hostname in groups['test']

  - name: replace the content in prod group
    copy:
      content: "Production"
      dest: /etc/issue
    when: inventory_hostname in groups['prod']
...
```

Q13. Rekey an existing Ansible vault as follows:
* Download the Ansible vault from "http://classroom.example.com/content/secret.yml"
* The current vault password is curabete
* The new vault password is newvare
* The vault remains in an encrypted state with the new password

Solution:

wget http://classroom.example.com/content/secret.yml

ansible-vault rekey --ask-vault-pass secret.yml
Vault password: curabete
New Vault password: newvare
Confirm New Vault password: newvare
Rekey successful

Note: After solving this ques in exam, always check the new password that you set. Because sometimes we do mistake in copy-pasting as it's not visible what we are pasting there.

Q14. Create user accounts. A list of users to be created can be found in the file called user_list.yml which you should download from "http://classroom.example.com/content/user_list.yml" and save to /home/admin/ansible/.
* Using the password vault created elsewhere in this exam, create a playbook called create_user.yml that creates user accounts as follows:
* Users with a job description of developer should be:
- created on managed nodes in the dev and test host groups assigned the password from the dev_pass variable and is a member of supplementary group devops
* Users with a job description of manager should be:
- created on managed nodes in the prod host group assigned the password from the mgr_pass variable and is a member of supplementary group opsmgr
* Passwords should use the SHA512 hash format. Your playbook should work using the vault password file created elsewhere in this exam.

INSTRUCTIONS:

In this question, we just have to create users as per following:

- use the user_list file as vars file to get username
- use vault.yml created in ques 9 as password file.(we can also use vault file as var file)
- write two tasks for creating user. One with condition job description = developer and another with condition job description = manager. (job description is given in user_list file)

Note: while running playbook, don't forget to provide vault password file (--vault-password-file=password.txt)

Solution:

```
wget http://classroom.example.com/content/user_list.yml

cat user_list.yml

users:
  - name: adam
    job: developer
  - name: gabriel
    job: manager
  - name: lucifer
    job: developer

vim create_user.yml

---
- name: playbook for create_user.yml
  hosts: all
  vars_files:
    - user_list.yml
    - vault.yml
  tasks:
    - name: create the groups
      group:
        name: "{{ item }}"
      state: present
      loop:
            - devops
            - opsmgr

    - name: create the users with developer job profile
      user:
```

```
    name: "{{ item.name }}"
    groups:  devops
    append: true
    password: "{{ dev_pass | password_hash('sha512') }}"
    state: present
  loop: "{{ users }}"
  when:
    - item.job == 'developer'
    - inventory_hostname in groups['dev'] or inventory_hostname in groups['test']

- name: create the users with manager job profile
  user:
    name: "{{ item.name }}"
    groups: opsmgr
    append: true
    password: "{{ mgr_pass | password_hash('sha512') }}"
    state: present
  loop: "{{ users }}"
  when:
    - item.job == 'manager'
    - inventory_hostname in groups['prod']
...
```

ansible-playbook create_user.yml --vault-password-file=password.txt

Q15. Create a playbook storage.yml for creating Logical volumes in all nodes according to following requirements.

* Create a new Logical volume named as 'data'
* LV should be the member of 'research' Volume Group
* LV size should be 1500M
* It should be formatted with ext4 filesystem.

-- If Volume Group does not exist then it should print the message "VG Not found"
-- If the VG can not acccomodate 1500M size then it should print "LV Can not be created with following size"
-- then the LV should be created with 800M of size.
-- Do not perform any mounting for this LV.

INSTRUCTION:

First clear the LVM concept before practicing this question.

In exam, vdb will be there in 4 nodes. Partitions and vg (volume group) will be already created. You just have to create lv (logical volume).

Out of these 4 nodes, one node will have vg with space less than 1500. (Probably node4)
So according to condition given in ques, here you have to create lv of size 800M and also print the message.

In one node (probably node5), vg will not be there. So for this node, in rescue part you have to print a message (debug) “VG Not Found”.

In our lab, for creating exam environment (partitions and vg), first attach a hard disk of size > 1500M. Then create partitions and vg. For creating these you can run the playbook “lvm.yml”.

In solution, we are using block and rescue and when for managing errors and conditions.

Step1: Write playbook till rescue – debug: lv_info (from this we can print errors that we will further use in when condition.)
Step2: Using those errors, write further playbook for debug and creating lvm with size 800m
Step4: Then in always write task for creating filesystem/format

[Only to do here for creating the scenario]

vim lvm.yml

```
---
- name: playbook for lvm creation
  hosts: all
  tasks:
    - name: create new partion
      parted:
        device: /dev/vdb
        number: 1
        state: present
        part_end: 2GB
      when: ansible_hostname == 'servera' or ansible_hostname == 'serverb' or
ansible_hostname == 'serverc'

    - name: create a volume group
      lvg:
        vg: research
        pvs: /dev/vdb1
      when: ansible_hostname == 'servera' or ansible_hostname == 'serverb' or
ansible_hostname == 'serverc'

    - name: create new partion
      parted:
```

```
      device: /dev/vdb
      number: 1
      state: present
      part_end: 1GB
    when: ansible_hostname == 'serverd'

  - name: create a volume group
    lvg:
      vg: research
      pvs: /dev/vdb1
    when: ansible_hostname == 'serverd'

...
```

Solution:

vim storage.yml

```
---
- name: playbook for storage.yml
  hosts: all
  tasks:
    - block:
        - name: Create a logical volume of 1500M
          lvol:
            vg: research
            lv:  data
            size: 1500m
          register: lv_info

      rescue:
        - debug:
            var: lv_info

        - debug:
            msg: " VG Not found"
          when: '"does not exist" in lv_info.msg'

        - debug:
            msg: "LV Can not be created with following size"
          when: '"insufficient free space" in lv_info.err'

        - name: Create a logical volume of 800M
          lvol:
```

```
      vg: research
      lv:  data
      size: 800m
    register: lv_info_new
    when: '"insufficient free space" in lv_info.err'

        - debug:
      var: lv_info_new

  always:
   - name: create filesystem
     filesystem:
           fstype: ext4
     dev: "/dev/research/data"
...
```

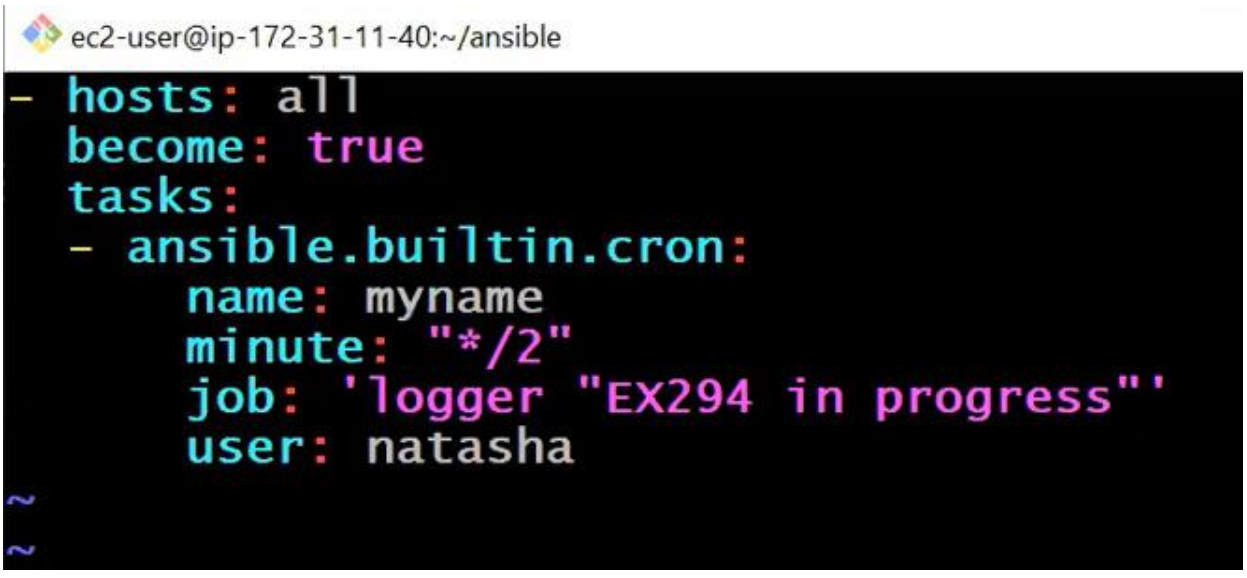
Check: -
ansible all -a 'lvdisplay'
ansible all -a 'blkid'

Q16) Configure a cron job

Create a playbook called /home/Catherine/ansible/cron.yml that runs on all managed hosts andcreates a cron job for user Natasha as follows:

The user Natasha must configure a cron job that runs every 2 minutes and executes logger
“EX294 in progress”

Solution –
cd /home/Catherine/ansible
vim cron.yml



 # ansible-navigator run -m stdout cron.yml --syntax-check cron.yml
 # ansible-navigator run -m stdout cron.yml

Q17)
Install a Collection

Install the following collection artifacts available from http://rhgls.sector1.example.com/materials to **control.sector1.example.com** as the user **Catherine**

redhat-rhel_system_roles-

1.16.2.tar.gzansible-posix-

1.4.0.tar.gz

community-general-4.3.0.tar.gz

The collections should be installed into the default collections directory **/home/Catherine/ansible/mycollections**

yum install wget -y
cd /home/Catherine/ansible/mycollection
podman login <registry name>
wget <url path>
podman pull <image path>
dnf install rhel-system-roles.noarch

Q18) Using a selinux role create a selinux.yml playbook to configure on all managed hosts to set the default target as permissive

Solution:

pwd
ansible-galaxy install linux-system-roles.selinux
vim playbook.yml
- name: creating a play for selinux
  hosts: all
  vars:
    selinux_status:
  roles:
    - linux-system-roles.selinux

ansible-navigator run -m stdout playbook.yml --syntax-check
ansible-navigator run -m stdout playbook.yml

TIPS FOR EXAM

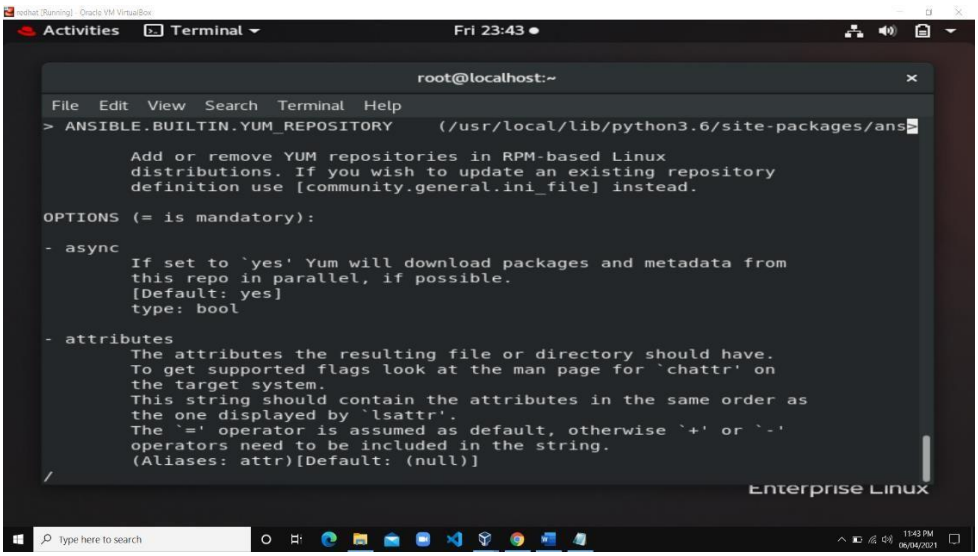
- **IF you forget any module name**
  You can search for module name using ansible-doc command.
  → ansible-doc –l | grep <keyword_related_to_module>
  EX: ansible-doc –l | grep yum
  By this approach you can find the module name.

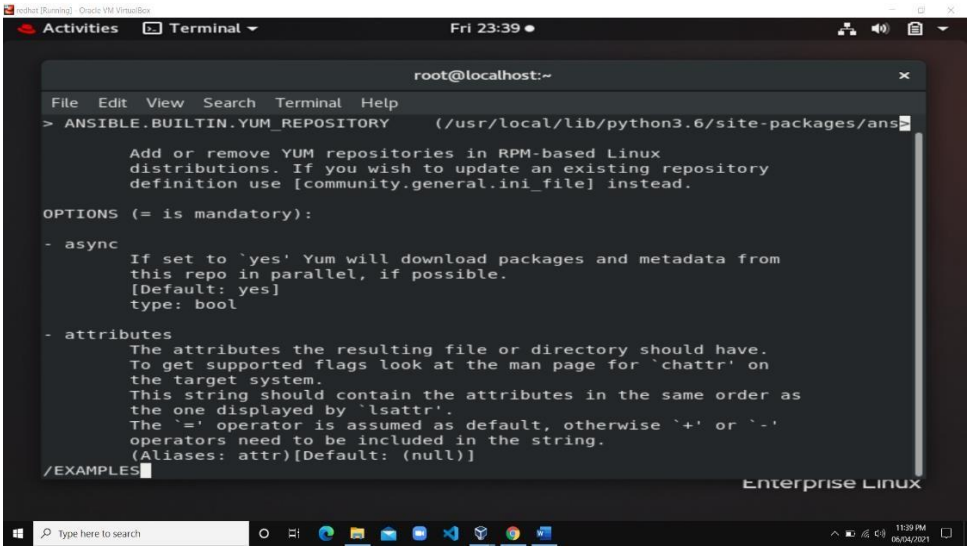
- **If you forget any arguments of module**
  You can see the documentation of any module using ansible-doc command.
  → ansible-doc <module_name>
  EX: ansible-doc yum_repository

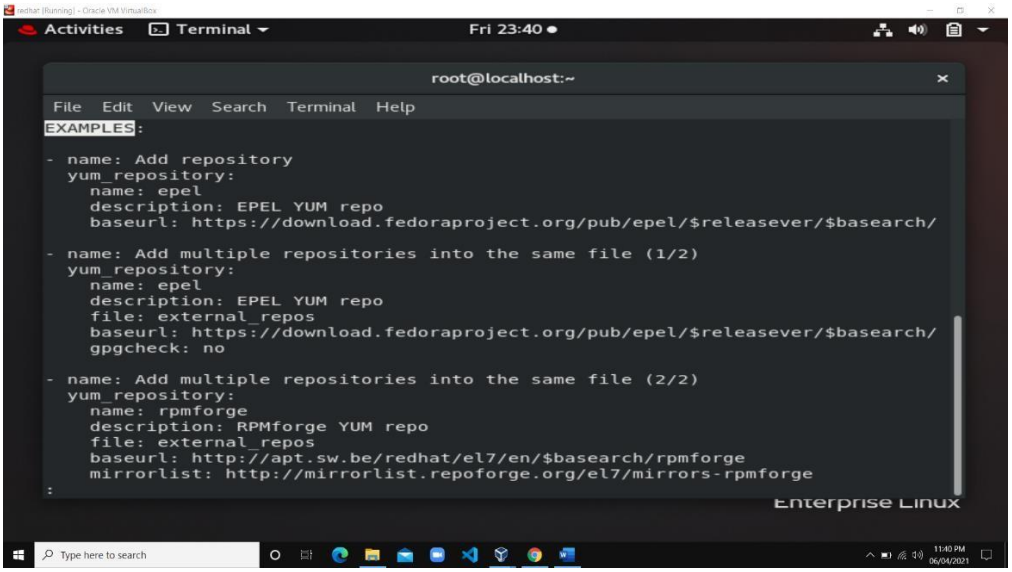
- **Search of specific keyword in the output of ansible-doc**
  Once the output of "ansible-doc module_name" command is displayed then you can search for any specific keywords or also you can directly jump to EXAMPLES.
  1. Press /
  2. Enter the pattern to search after /
  3. Press Enter to perform the search
  4. Press n to find the next occurrence or N to find the previous occurrence.

  Note: You can use similar approach in vim editor also.

Examples Screenshot is attached in next page.







- You can use two terminals in exam. One for creating playbooks and another for testing or checking ansible-doc.

- You can also create backup of main folder in /temp folder as taught by sir.